Figure 5: A simple calculator

Of course, it is possible to write two different event handlers for *btn1* and *btn2*. But sometimes we may want to perform the same tasks (with some minor differences) when *btn1* and *btn2* are clicked. On such occasions, we can use the method mentioned above.

A simple calculator

Now let's check whether the knowledge we gained is useful or not. Here is the code for a simple calculator. It seems to be self-explanatory. Some comments are also provided, where necessary.

```
import gtk
wnd = gtk.Window()
vbox = gtk.VBox()
hbox = gtk.HBox()
txt1 = gtk.Entry()
txt2 = gtk.Entry()
btn_add = gtk.Button('+')
btn_sub = gtk.Button('-')
btn_mul = gtk.Button('X')
btn_div = gtk.Button('/')
lbl_res = gtk.Label()
def on_btn_click(widget):
     n1 = float(txt1.get_text()) # convert string to a decimal
no.
     n2 = float(txt2.get_text()) # convert string to a decimal
no.
     operation = widget.get_label()
     res = 0
     if(operation == '+'):
       res = n1+n2
     elif(operation == '-'):
       res = n1-n2
     elif(operation == 'X'):
       res = n1*n2
     else: # division
       if(n2 == 0): res="Division by zero not possible."
       else: res = n1/n2
     lbl_res.set_label(str(res));
hbox.add(btn_add)
hbox.add(btn_sub)
hbox.add(btn_mul)
hbox.add(btn_div)
vbox.add(txt1)
vbox.add(txt2)
vbox.add(hbox)
vbox.add(lbl_res)
wnd.add(vbox);
vbox.set_spacing(10) # spacing between widgets
wnd.set_title("Simple Calculator")
wnd.set_size_request(300, -1)
wnd.set_border_width(10)
wnd.show_all()
btn_add.connect("clicked", on_btn_click)
btn_sub.connect("clicked", on_btn_click)
btn_mul.connect("clicked", on_btn_click)
btn_div.connect("clicked", on_btn_click)
wnd.connect("destroy", gtk.main_quit)
gtk.main()
```

GTK+ 3

As mentioned earlier, the version of GTK+ we used in this tutorial is 2.x. GTK+ 3 offers a lot more. As I said, we didn't use it because it may still be unavailable for some users (who use old distros). However, it is time to upgrade. There are not many changes in the code. Consider the following code as the starting point of your migration to version 3.

```
from gi.repository import Gtk
wnd = Gtk.Window()
lbl = Gtk.Label("Hello, World!")
wnd.add(lbl)
wnd.show_all()
wnd.connect("destroy", Gtk.main_quit)
Gtk.main()
```

Not a headache, is it?

Further exploration

In this tutorial, we made use of a few widgets, adjusted some of their properties, and got introduced to basic event handling. But PyGTK contains many more widgets, and each of them has a lot of methods, properties and events. How can you explore them? Just visit:

https://developer.gnome.org/pygtk/stable/

Here, you'll find the detailed reference material of PyGTK. Also, you can download and install offline versions of Devhelp and PyGTK documentation. END

By: Nandakumar Edamana

Being a free software (free as in freedom) user, developer, and activist, Nandakumar has created a handful of software packages, which are available at nandakumar.co.in. Sammaty Election Engine, one of the packages developed by Nandakumar, became a popular choice among thousands of schools in Kerala. He can be contacted at *nandakumar@nandakumar.co.in* or *nandakumar96@gmail.com*